

YOUTUBE SYSTEM DESIGN

- DECIDING REQUIREMENTS
 - 1.Functional Requirements
 - 2.Non Functional Requirements
 - CAPACITY ESTIMATION
 - 3.DAU-MAU
 - 4.Throughput
 - 5.Storage
 - 6.Memory
 - 7.Network/Bandwidth Estimation
 - API DESIGN
 - 8.API Design-Upload Content
 - 9.API Design-Stream Content
 - HIGH LEVEL DESIGN
 - 10.High Level Design-Upload content
 - 11.High Level Design-Upload content-2
 - 12.High Level Design-Stream content
 - 13.High Level Design-Content Processor Workflow Engine
 - DEEP DIVE INSIGHTS
 - 14.Deep Dive Insights-Database Selection
 - 15.Deep Dive Insights-Database Modeling
 - 16.Deep Dive Insights-HLS Encoding
-

DECIDING REQUIREMENTS:

Functional Requirements

There will be two types of users:

- 1. **Viewers:** One who consumes the content.
- 2. **Content Creators:** One who creates the content.

Functional Requirements for Viewers

Requirement	Description
Streaming	User can stream any TV show or movie.
Device Compatibility	The application should be supported on all devices.

Functional Requirements for Content Creators

Requirement	Description
Upload Content	Content Creators should be able to upload movies/TV shows. After successful upload, they should get a notification.

Non-Functional-Requirements

For Viewers

Requirement	Description
Low Latency	When a user watches a movie or TV show, the experience should be smooth. Any buffering or lagging will spoil the experience.

Scalability	The platform should be able to handle a large number of users streaming many movies/TV shows simultaneously.
User Experience	Users should get the best possible quality while streaming. For example, if a user's internet is strong, they shouldn't receive poor quality (240p) streaming.
Availability	The system should be highly available with 99.9999% uptime.

For Content Creators

Requirement	Description
Scalability	The platform should be able to handle many content creators uploading numerous movies/TV shows at the same time.
Security	The platform should ensure the security of the content uploaded by creators to prevent unauthorized access or piracy.
Storage Reliability	The platform should provide reliable storage to ensure the uploaded content is safely stored and remains available without disappearing.

CAPACITY ESTIMATION:

DAU-MAU

How many users will be using your software?

- **Daily Active Users (DAU):** 100 million
- **Monthly Active Users (MAU):** 2.5 billion

Throughput

1. Write Requests

Write Operation:

Uploading videos is the only way to write (add data) to the system.

Assumption:

Most users are viewers rather than creators. We assume that 1 out of 250 users uploads a video daily.

Calculation:

- **Total Daily Active Users (DAU):** 100 million
- **Fraction of Users Uploading Videos:** $(1/250)$
- **Write Requests per Day:**

$$(1/250) \times 100,000,000 = 0.4 \text{ million write requests per day}$$

2. Read Requests

Read Operation:

Watching videos is the primary way to read data from the system.

Assumption:

An average user watches 10 videos per day.

Calculation:

- **Total Daily Active Users (DAU):** 100 million
- **Videos Watched per User:** 10
- **Read Requests per Day:**

$$100,000,000 \times 10 = 1 \text{ billion read requests per day}$$

Summary

Operation	Calculation	Result
Writes	$(1/250) \times 100 \text{ million}$	0.4 million requests/day

Reads	100 million × 10	1 billion requests/day
-------	------------------	------------------------

Storage

Video Data

Assumptions

- **Average Size of a Video:** 600 MB
- **Daily Uploads (from throughput estimation):** 0.4 million requests per day

Storage Calculations

1. **Daily Storage Requirement:**
 $600 \text{ MB} \times 0.4 \text{ million requests/day} = \mathbf{240 \text{ TB/day}}$
2. **10-Year Storage Requirement:**
 $240 \text{ TB/day} \times 365 \text{ days} \times 10 \text{ years} = \mathbf{876 \text{ PB}}$

Summary

Metric	Calculation	Result
Daily Storage	$600 \text{ MB} \times 0.4 \text{ million requests/day}$	240 TB
10-Year Storage	$240 \text{ TB/day} \times 365 \text{ days} \times 10 \text{ years}$	876 PB

Memory

Overview

By memory, we refer to the **cache memory size** required for faster data access.

Why Cache Memory?

Accessing data directly from the database takes time. To speed up data retrieval, cache memory is used.

Cache Memory Requirement Calculation

- **Daily Storage Requirement:** 240 TB/day
- **Cache Requirement (1% of Daily Storage):**
 $0.01 \times 240 \text{ TB} = \mathbf{2.4 \text{ TB/day}}$

Scalability

The memory size should scale as the system grows to accommodate increasing storage and data access demands.

Network and Bandwidth Estimation

Overview

Network/Bandwidth estimation helps us determine the amount of data flowing in and out of the system per second.

Data Flow Estimations

Ingress (Data Flow Into the System)

- **Data Stored per Day:** 240 TB/day
- **Calculation:**
 $240 \text{ TB} \div (24 \times 60 \times 60) = \mathbf{2.7 \text{ GB/s}}$
- **Result:** Incoming Data Flow = **2.7 GB/s**

Egress (Data Flow Out of the System)

- **Total Read Requests per Day:** 1 billion
- **Average Video Size:** 600 MB

- **Daily Outgoing Data:**

1 billion × 600 MB = **600 PB/day**

- **Calculation:**

600 PB ÷ (24 × 60 × 60) = **7 TB/s**

- **Result:** Outgoing Data Flow = **7 TB/s**

Summary

Type	Calculation	Result
Ingress (Data Flow In)	240 TB ÷ (24 × 60 × 60)	2.7 GB/s
Egress (Data Flow Out)	600 PB ÷ (24 × 60 × 60)	7 TB/s

API DESIGN:

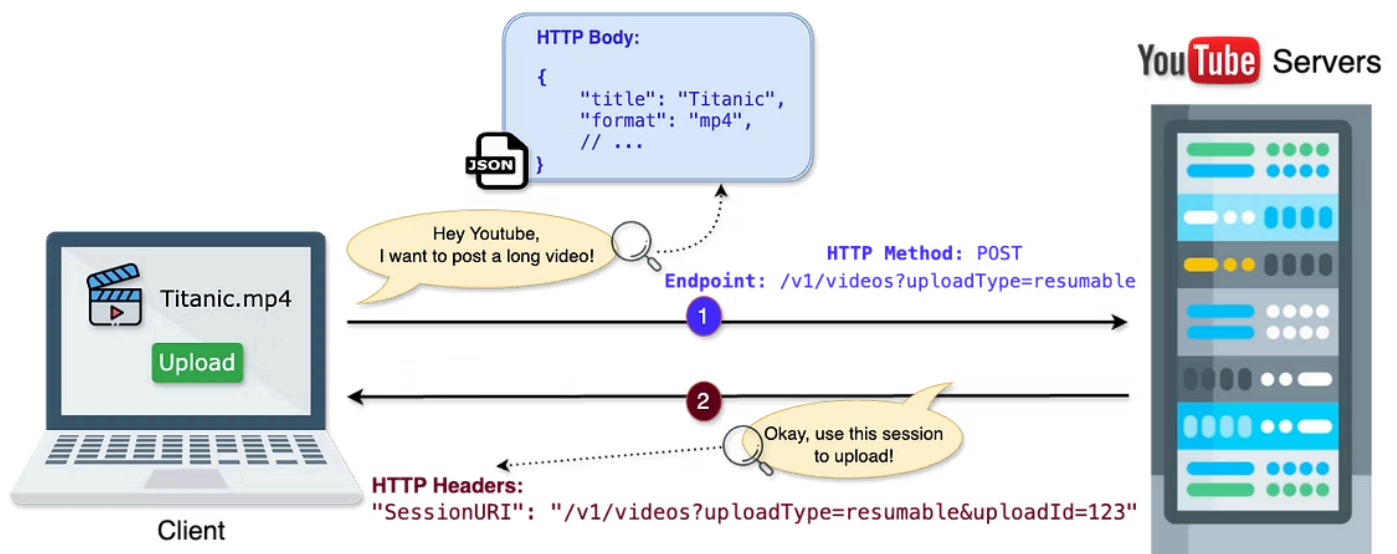
API-DESIGN Upload Content

Let's understand how a client (content creator) uploads content to YouTube and what APIs are involved.

Since videos can be very large (could be 10 minutes or even 2 hours), it's not feasible to upload a video in a single request. Multiple requests are sent to upload small chunks of the video.

Additionally, there is video metadata associated with the video, such as the video title, creator ID, format, etc.

When we press the upload button, initially a request is sent to add the metadata associated with the video to the server. Here are the details of the initial request.



Initial Request (Add Video Metadata)

HTTP Method

This tells the server what action to perform. Since we want to create something new on the server (metadata for the new video), we use the **POST** action.

Endpoint

This tells the server where to perform that action. Since we are creating video metadata, we use the **/v1/videos** endpoint of the server.

Note

Since we also need to upload the actual video after the metadata, we include **uploadType=resumable** in our endpoint. This indicates that we are uploading a large media file chunk by chunk. It is useful because if the connection drops while uploading this large file, we should be able to resume the upload from that point. In summary, **uploadType=resumable** tells the server to provide a "resumable" URL back that we can use to upload as well as resume the upload, if needed.

HTTP Body

We tell the server to create metadata for the video, but we haven't provided the details of the metadata yet. This information is sent in the request body.


```

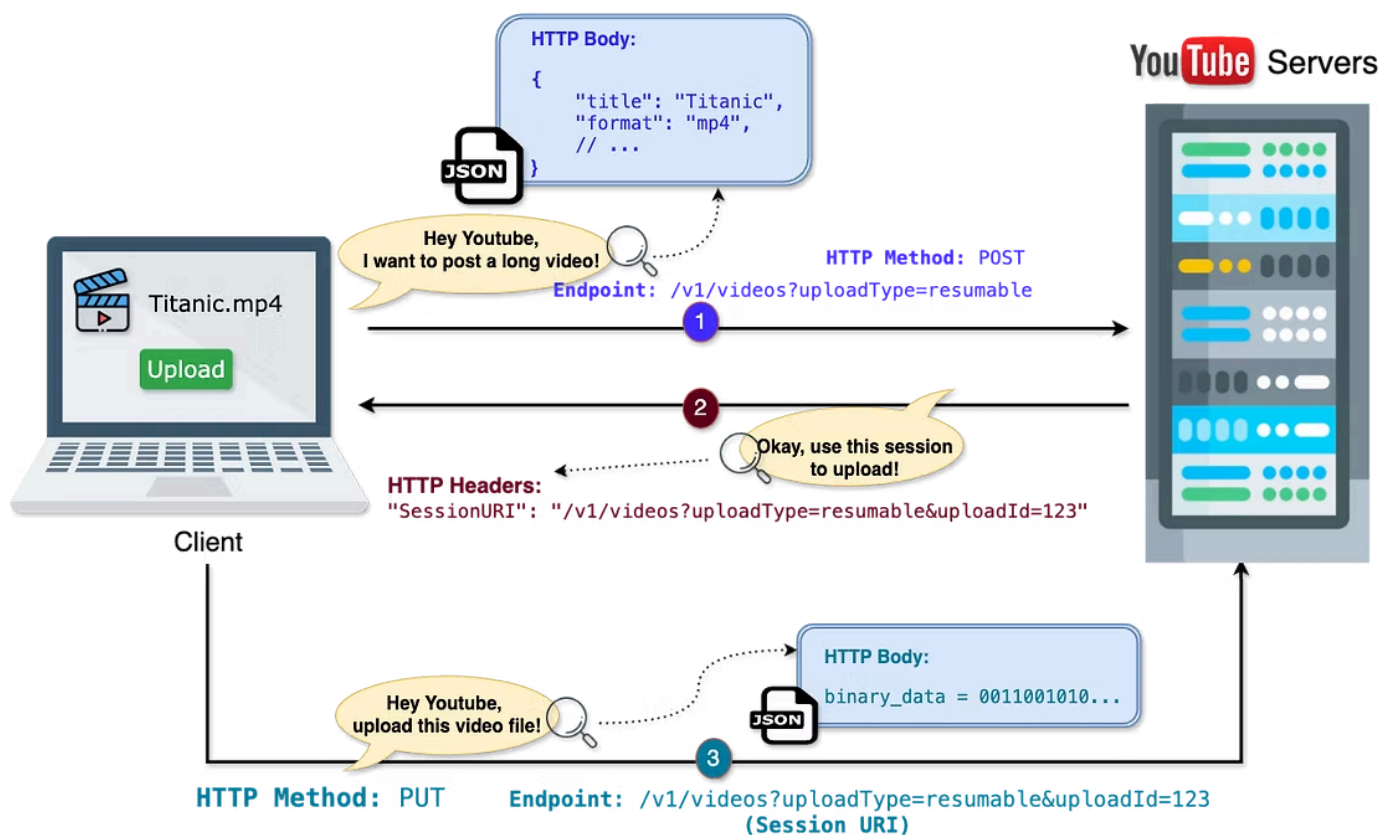
{
  "title": "Your video title",
  "format": "The format of your video"
  // ...
}

```

In response to this request, YouTube's server provides us with a resumable URL. We can use this URL to upload the video, and if the connection drops, we can resume the upload from that point. This resumable URL is called a session.

When we upload the actual video, we use this same session URL. Here are the details of the actual upload request:

Final Request (Upload Video)



HTTP Method

This tells the server what action to perform. We use the **PUT** action to upload the actual video data to the given session URL.

Endpoint

This tells the server where to perform that action. We use the session URI provided by the server in the response to the first request. The **uploadId** in the URI helps the server identify which video's session it is, even when resuming the upload.

HTTP Body

The body of the **PUT** request contains the binary data of the video file.

API-DESIGN: Stream Content

Let's understand how a client streams content on YouTube and what APIs are involved.

Since videos can be very large (could be 10 minutes or even 2 hours), it's not feasible to get the whole video in a single request. Multiple requests are sent to get small chunks of the video. These chunks are stored at different locations on the server.

To stream the entire video, the client needs to know these locations. All these locations are saved in a file called a **manifest file**. So when the client first requests to watch a video, the server sends the manifest file. Once the client gets the manifest file, it uses it to fetch the different chunks from the server (this is streaming).

Flow Overview

Here's how the flow looks:

1. Watch Request

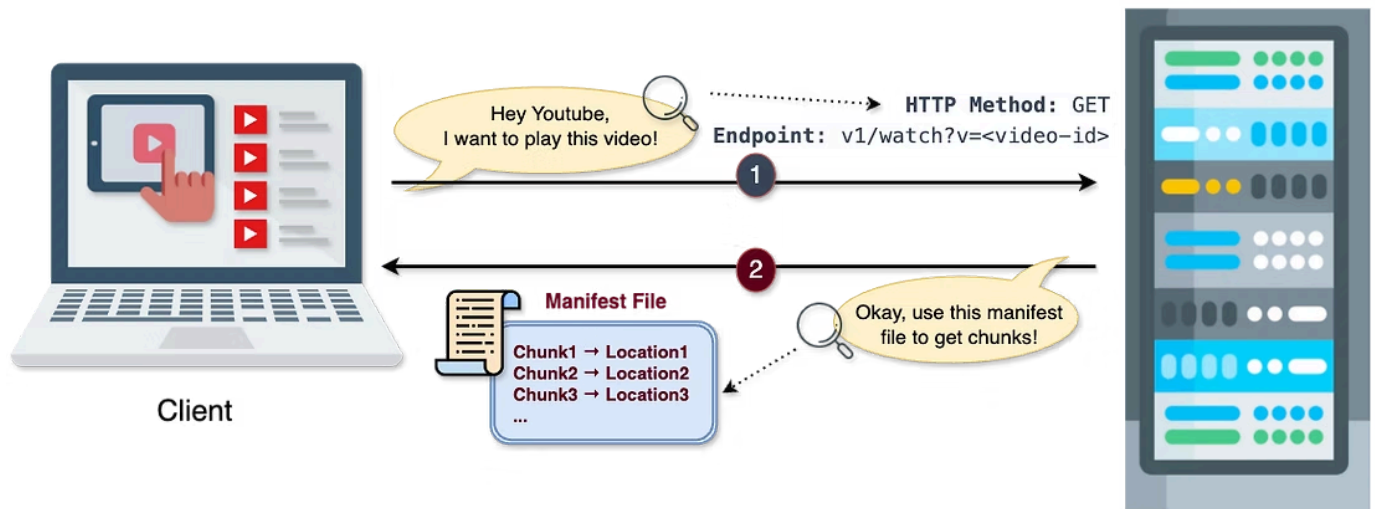
When we open a video, the first request that goes to the server is a watch request, telling the server that the client wants to watch the video.

2. Server Response (Manifest File)

The server responds with the manifest file.

3. Fetching Chunks

The client uses the manifest file to fetch different chunks from the server to stream the video.



When we open a video, the first request that goes to the server is a **watch request**, telling the server that the client wants to watch the video. The server responds with the manifest file. Here are the details about the request.

Initial Watch Request

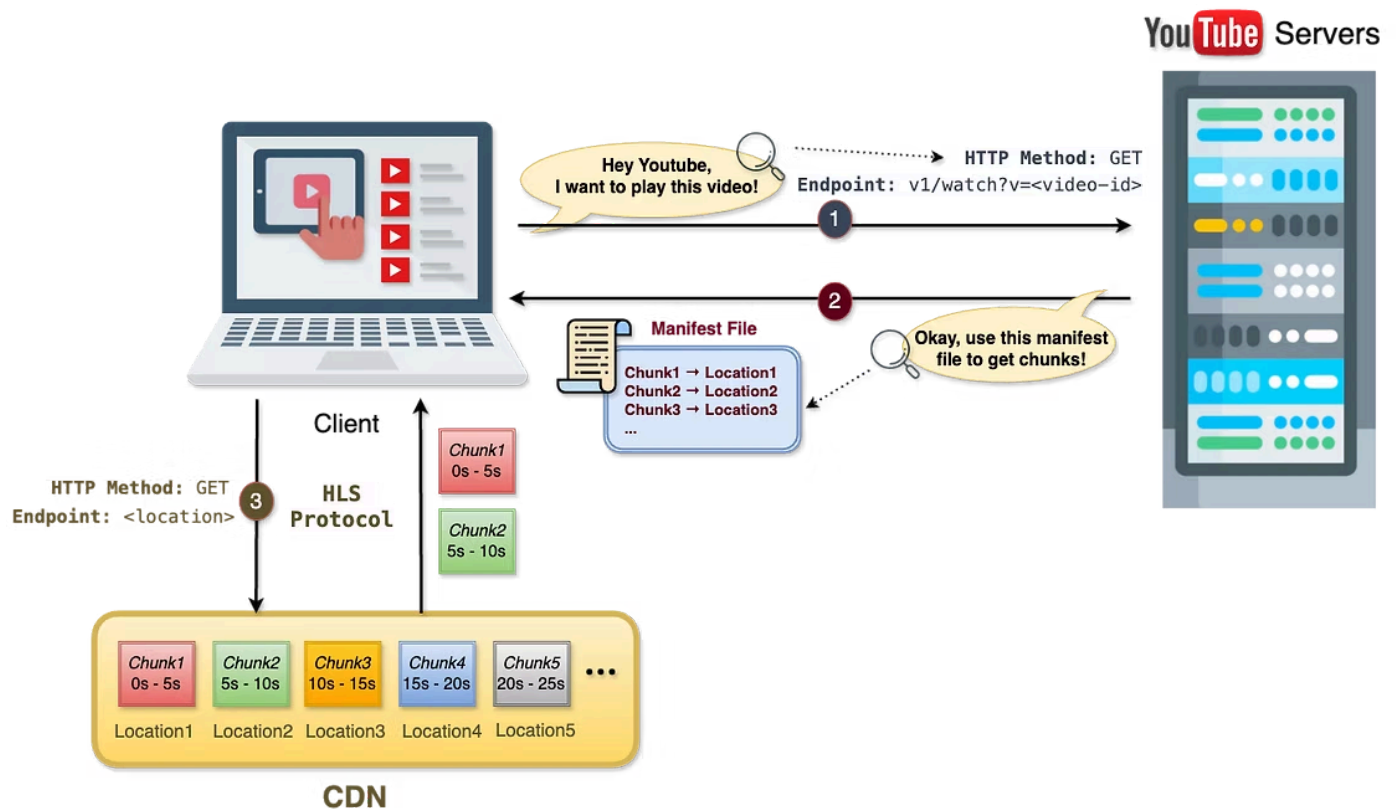
HTTP Method

This tells the server what action to perform. Since we want to watch (get) a video, we use the **GET** action.

Endpoint

This tells the server where to perform that action. We use the **/v1/watch** endpoint to tell the server we want to watch a video. The server sends the manifest file back.

Now the client has the manifest file which contains the locations of the video chunks. The client uses these locations to start streaming the video chunk by chunk. This is how the overall flow looks with the streaming request.



Streaming Request

The locations in the manifest file are actually the **CDN server locations** where the chunks are stored. Simply put, a **CDN** is a server that makes it easy to load large assets (like videos and media).

HTTP Method

This tells the server what action to perform. Since we are getting video chunks, we use the **GET** action.

Endpoint

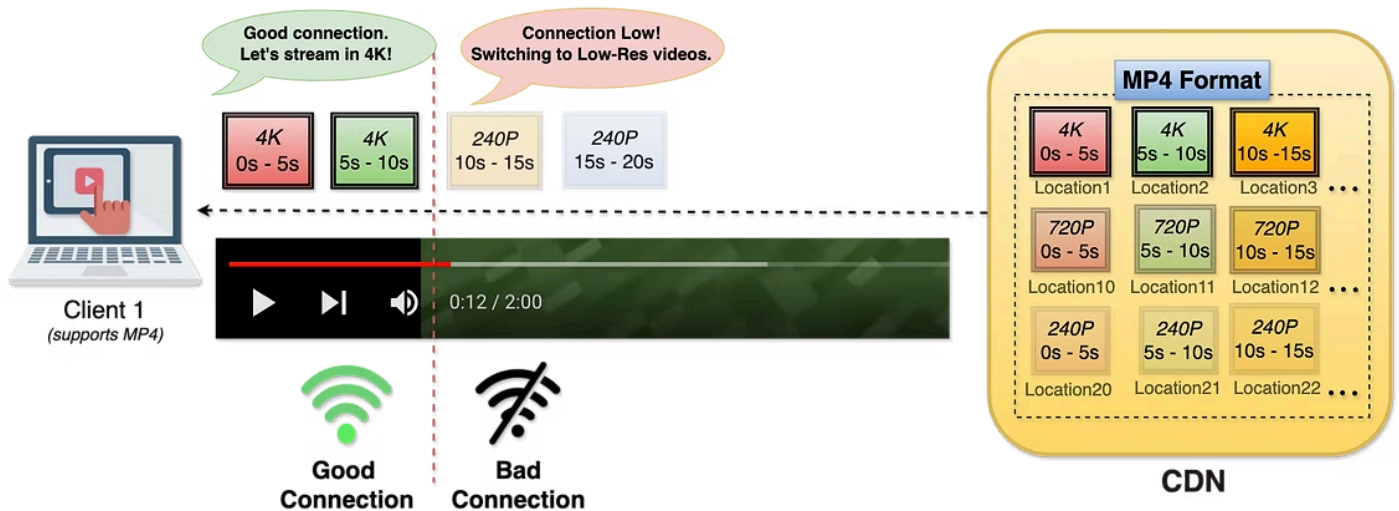
This tells the server where to perform that action. Since we are getting the video chunks from respective locations, we use the locations (provided in the manifest) as the endpoint.

HLS Protocol

We are using something called the **HLS Protocol** for getting video chunks from the CDN. **HLS (HTTP Live Streaming)** is a very popular protocol used for streaming.

Benefits of HLS

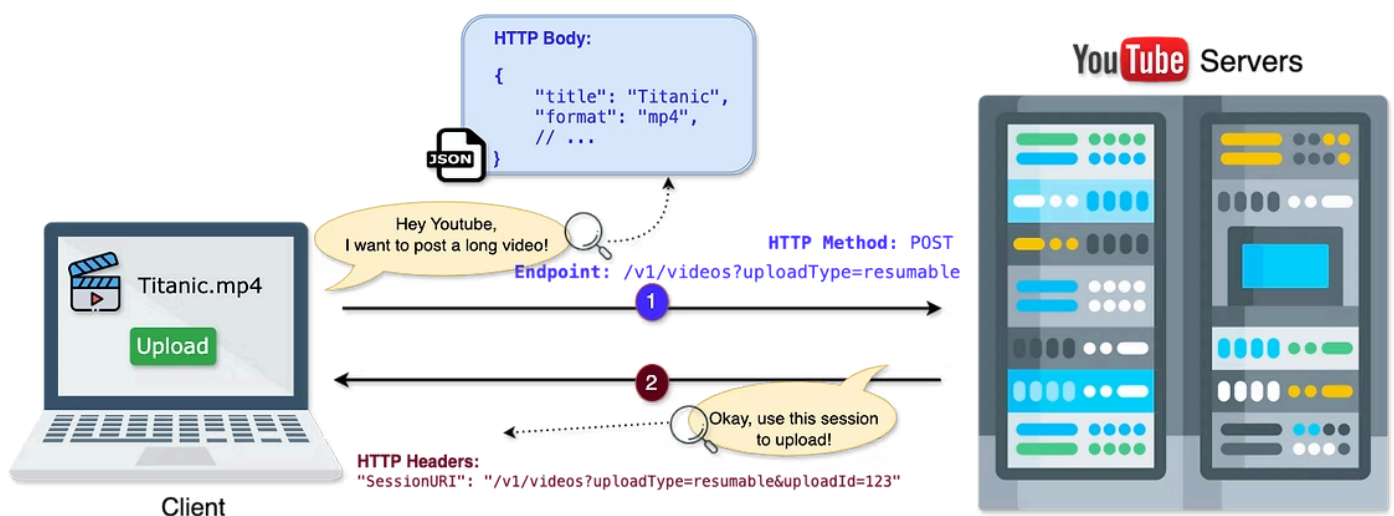
- **Adaptive Streaming:** The quality of the video can adjust based on the user's internet speed.
 - **Fast internet:** High-quality video chunks are streamed.
 - **Slow internet:** Video quality drops to prevent buffering, ensuring uninterrupted playback.



HIGH LEVEL DESIGN:

HIGH-LEVEL-DESIGN :Upload Content

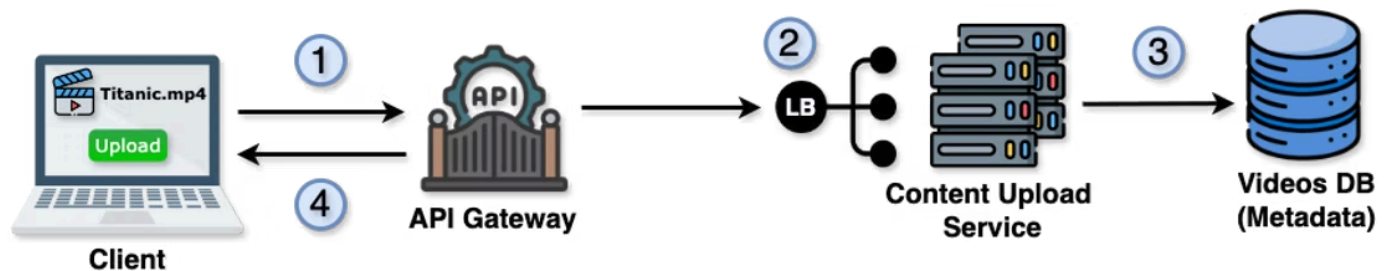
As we saw in the API design, the video upload process involves two main steps. The first request uploads the video metadata, and the server responds with a session URL.



High-Level Process

Let's understand this with a high-level diagram. Refer to the steps in the diagram.

1. When the client clicks the upload button, the first request goes to the **API Gateway**.
2. The **API Gateway** handles incoming requests and routes them to the **Content Upload Service** via the **Load Balancer**.
3. The **Content Upload Service** adds video metadata (title, format, etc.) to the **Videos DB**.
4. The client receives a successful confirmation along with a **session URL**.



This **session URL** is then used to upload the actual video data in chunks. Let's see how that happens next.

HIGH LEVEL DESIGN: Upload Content (Continued)

Once we have the session URL, we start uploading the video using that URL. Here's how the flow works (refer to the steps in the diagram):

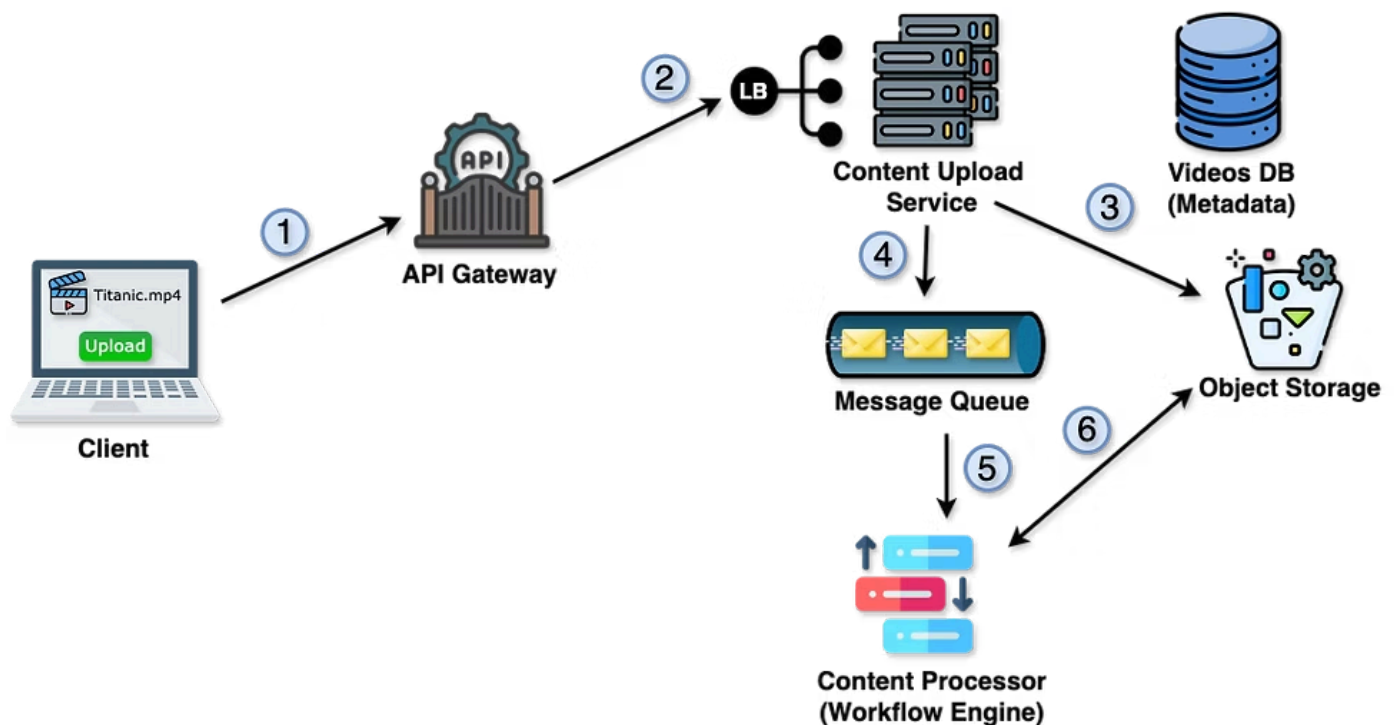
1. The client uses the session URL as an endpoint and sends a **PUT** request. This request contains the actual video content.
2. The API Gateway handles incoming requests and routes them to the Content Upload Service via the Load Balancer.
3. The Content Upload Service stores the video content in Object Storage.
4. When the entire video is uploaded to Object Storage, the Content Upload Service adds an event to the Message Queue. This event contains the video ID.

5. The Content Processor, also known as the Workflow Engine, pulls this event from the Message Queue for further processing. It's called a Workflow Engine because it runs a series of steps one after the other.

6. Workflow Operation

- **Step 1:** The Content Processor uses the event to retrieve the video from Object Storage.
- **Step 2:** Then, it breaks the large video file into smaller chunks.
- **Step 3:** Each chunk is converted into different formats and different qualities/resolutions. For example, a 1-minute video is broken into 10 chunks (6 seconds each). Each chunk is then converted into various formats (e.g., MP4, MOV) and resolutions (e.g., 4K, 720P, 240P).

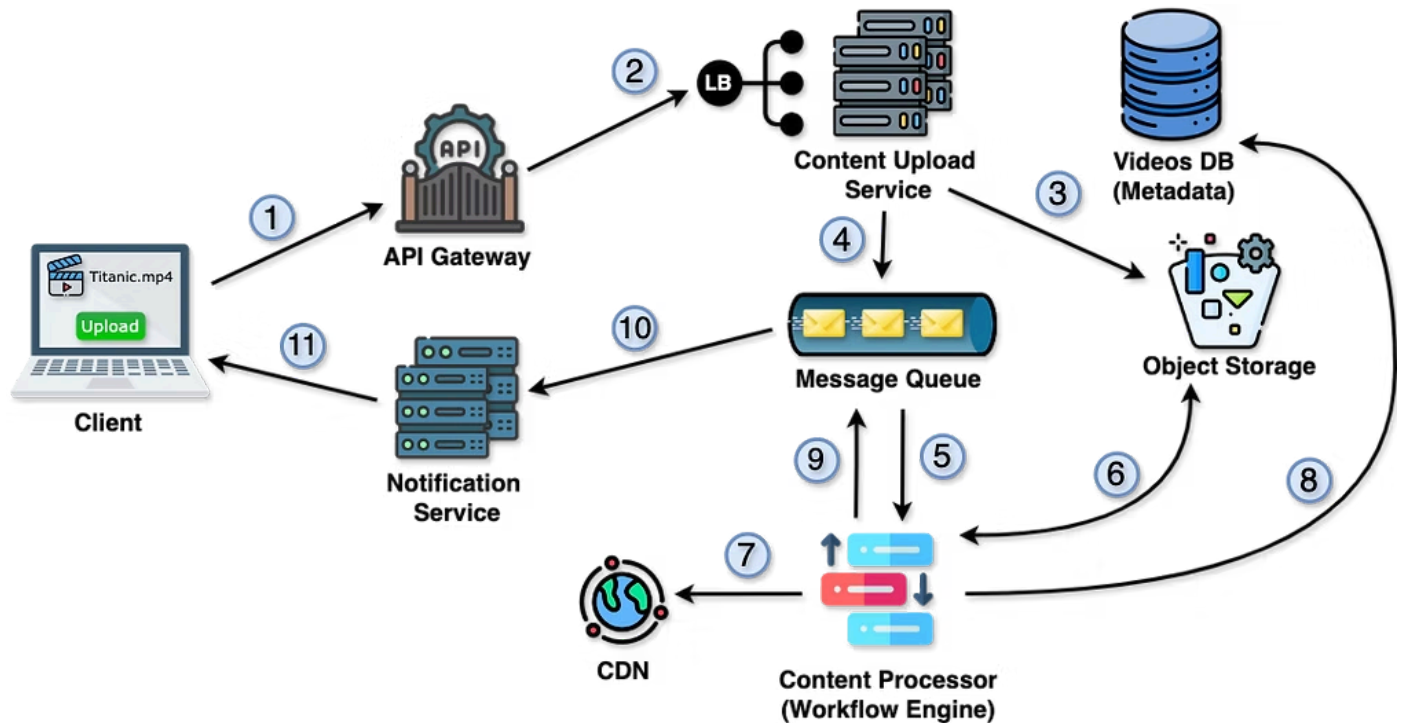
This process ensures the video can be accessed in different formats and qualities. We will discuss this workflow in detail in the next part.



7. The Content Processor uploads these smaller chunks (in different formats and qualities) to the CDN. Uploading to the CDN allows clients to access the videos faster.

8. Each chunk uploaded to the CDN has a location URL. The Content Processor saves these locations in the Videos DB.

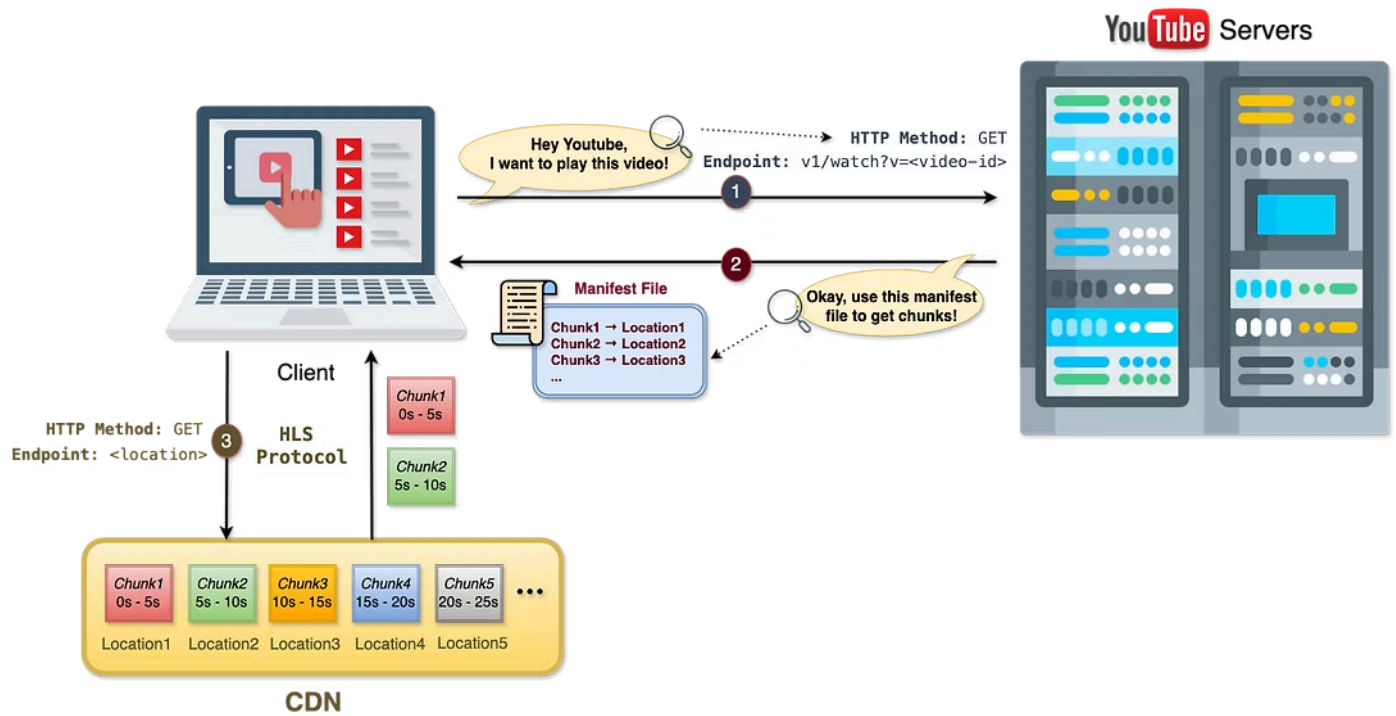
9. Since the video processing is complete, the Content Processor adds an event to the Message Queue to signal that the video processing is done.
10. The Notification Service pulls this event from the Message Queue for further processing.
11. Finally, the Notification Service sends a notification to the client saying the video upload process is complete.



HIGH LEVEL DESIGN :Stream Content

As we discussed during the API design, video streaming involves three main steps:

1. The client sends a request to the server indicating that it wants to play a specific video.
2. The server responds with a manifest file. This file contains the locations of the small video chunks.
3. The client streams these small video chunks from the CDN using the HLS protocol.



When the client sends a request to the server indicating that it wants to play a specific video, the server returns back the manifest file. Additionally, it returns other necessary video metadata like the title, creator ID, description, etc.

The manifest file from the server looks something like this. It lists the locations of video chunks in different formats and qualities for that video. The client then uses this file to request the video chunks.



Chunk	Parent Video	Format	Quality	Location
Chunk1 (0s - 5s)	Video1	MP4	4K	Location1
Chunk2 (5s - 10s)	Video1	MP4	4K	Location2
...				
Chunk10 (0s - 5s)	Video1	MP4	720P	Location10
...				
Chunk50 (0s - 5s)	Video1	MOV	4K	Location50

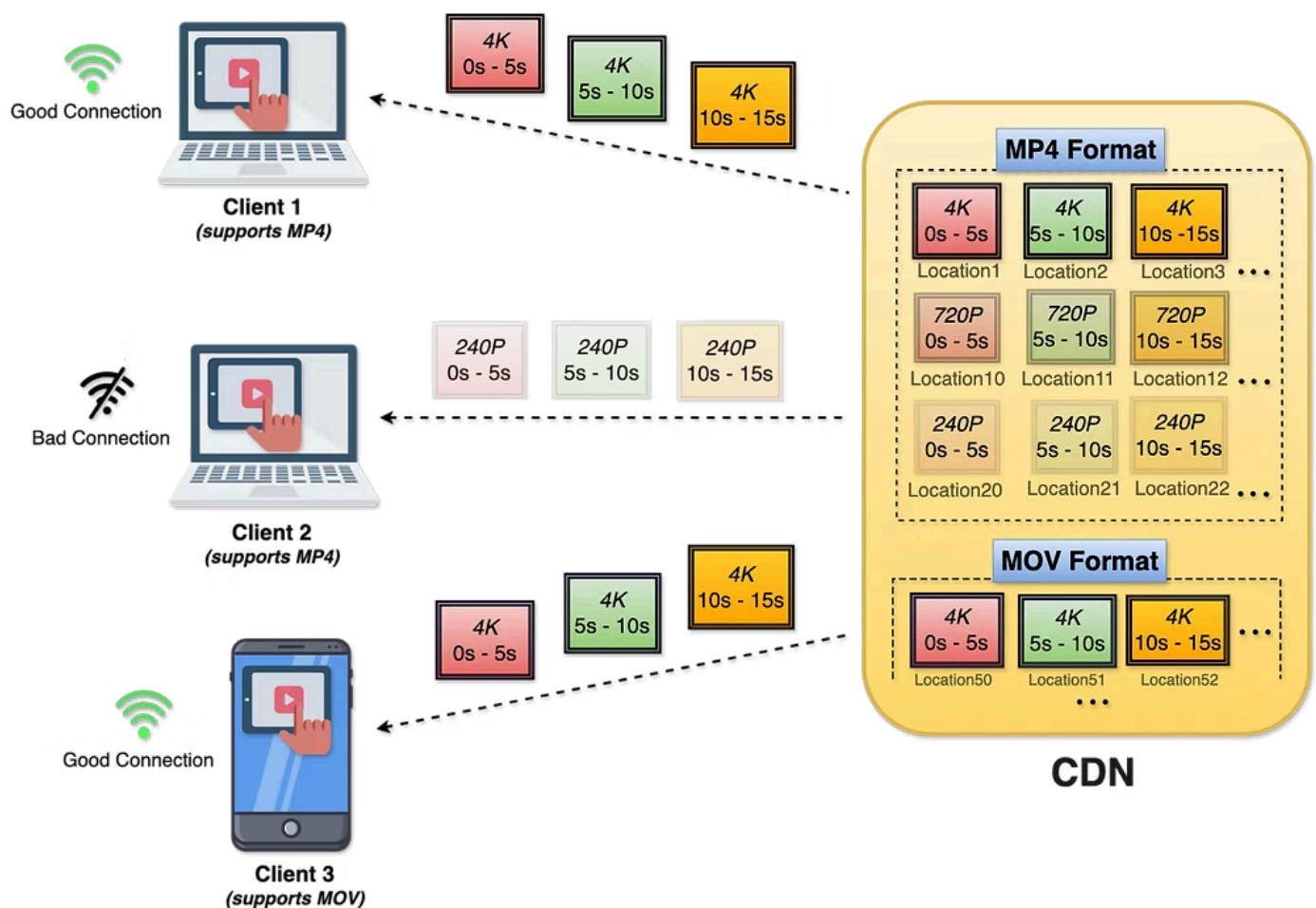
When the client requests video chunks from the CDN, the CDN delivers these chunks to the client via the **HLS protocol**.

Different devices support different video formats:

- When a client requests **MP4** format, it receives MP4 chunks.
- When a client requests **MOV** format, it receives MOV chunks.

Adaptive Quality Based on Network Strength

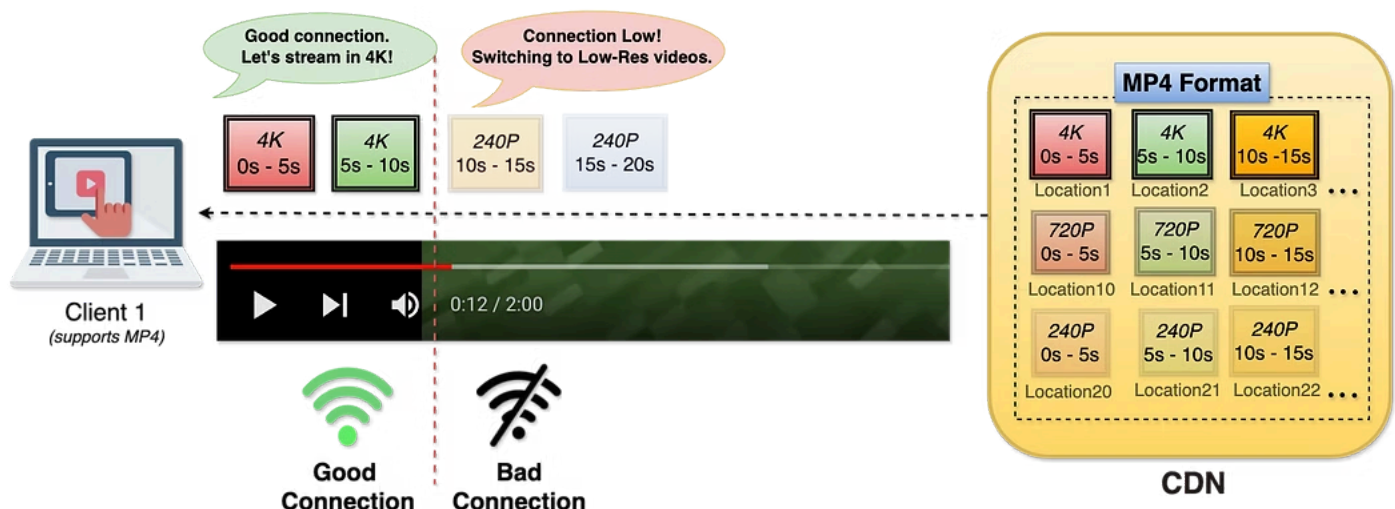
- **Weak Connection:** The client requests lower-quality chunks (e.g., **240P**).
- **Strong Connection:** The client requests higher-quality chunks (e.g., **4K**).



A common scenario occurs when the internet connection fluctuates between strong and weak:

- When the connection is strong, the client requests **high-quality** chunks.
- When the connection weakens, the client requests **lower-quality** chunks.

This adaptive behavior ensures a smooth viewing experience. This process is known as **Adaptive streaming**, and the **HLS protocol** supports it very well. This is why HLS is widely used during video streaming.



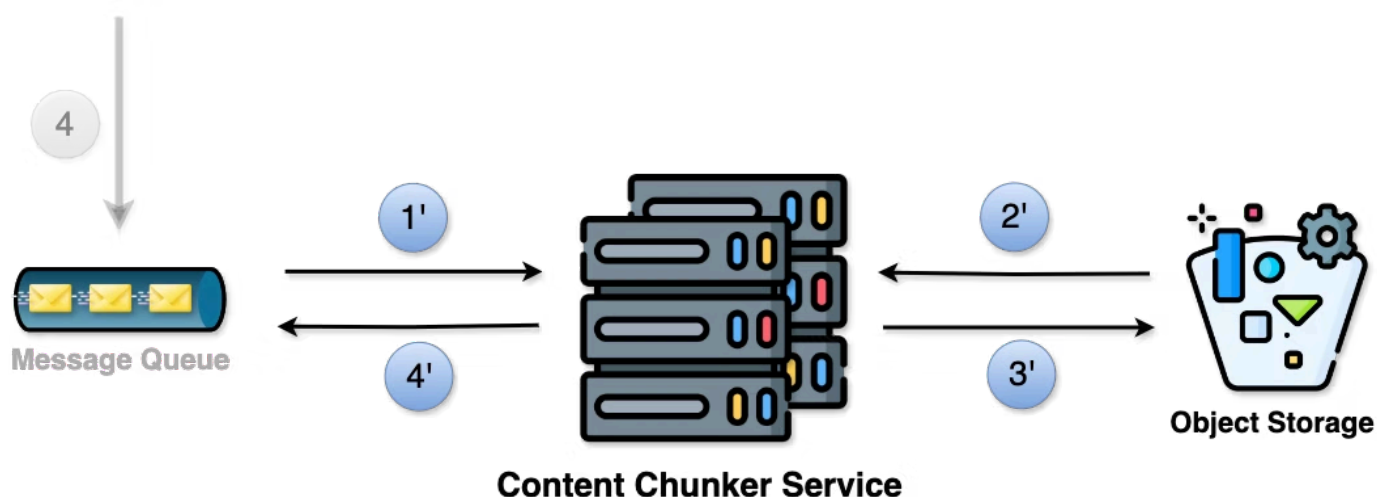
HIGH LEVEL DESIGN: Content Processor Workflow Engine

Let's dive deeper into the Content Processor (Workflow Engine). As mentioned, this is a workflow engine that runs a series of steps. Here are the steps:

- **Content Chunker Service**
- **Format Converter Service**
- **Quality Converter Service**
- **CDN Uploader Service**

Content Chunker Service

As we saw earlier, in the 4th step, an event is added to the Message Queue with the video ID. The Content Chunker Service grabs this video using the ID and breaks it into smaller chunks. This is necessary because it breaks the video into smaller chunks that can be processed in parallel. Also, it will make streaming efficient.

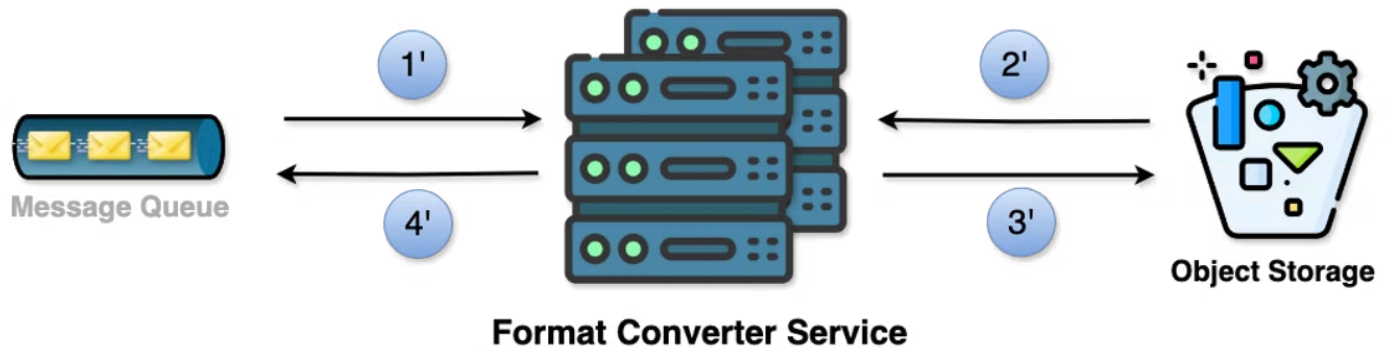


Here are the 4 precise steps:

1. **Retrieve the event from the Message Queue.**
2. **Use the video ID from the event to get the video from Object Storage and break it into smaller chunks.**
3. **Upload the chunks to Object Storage.**
4. **For each uploaded chunk, create an event with the chunk ID and add it to the Message Queue.**

Format Converter Service

After chunking, the **Format Converter Service** converts these chunks into different formats. This is necessary because converting the chunks into different formats ensures compatibility with various devices and platforms.



Steps Involved

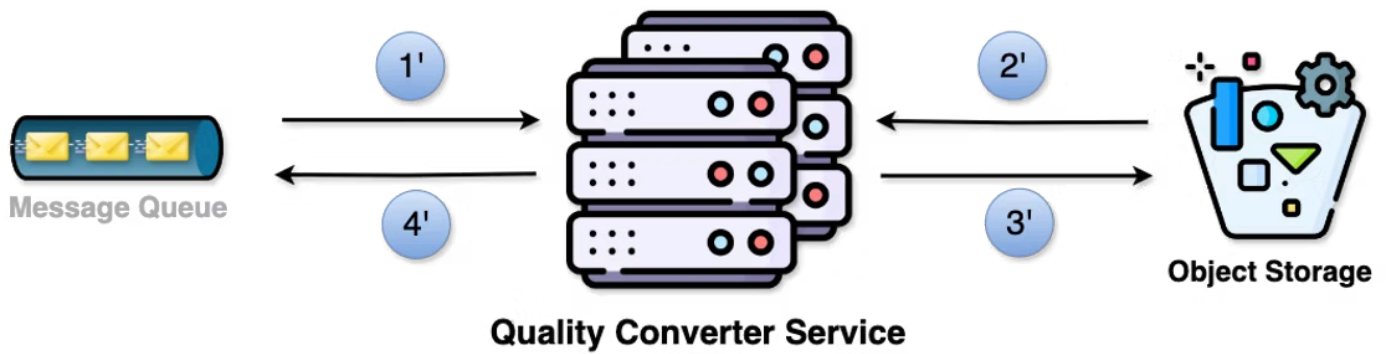
1. **Retrieve the event from the Message Queue.**
2. **Use the chunk ID from the event to get that chunk from Object Storage.**
Then it converts the chunk into different formats (e.g., MP4, MOV).
3. **Upload the converted chunks to Object Storage.**
4. **For each uploaded chunk, create an event with the chunk ID and add it to the Message Queue.**

Quality Converter Service

After converting to different formats, the Quality Converter Service converts these chunks into different quality levels. This is necessary because converting the chunks into different quality levels provides a smooth viewing experience as per the user's internet speed.

Example:

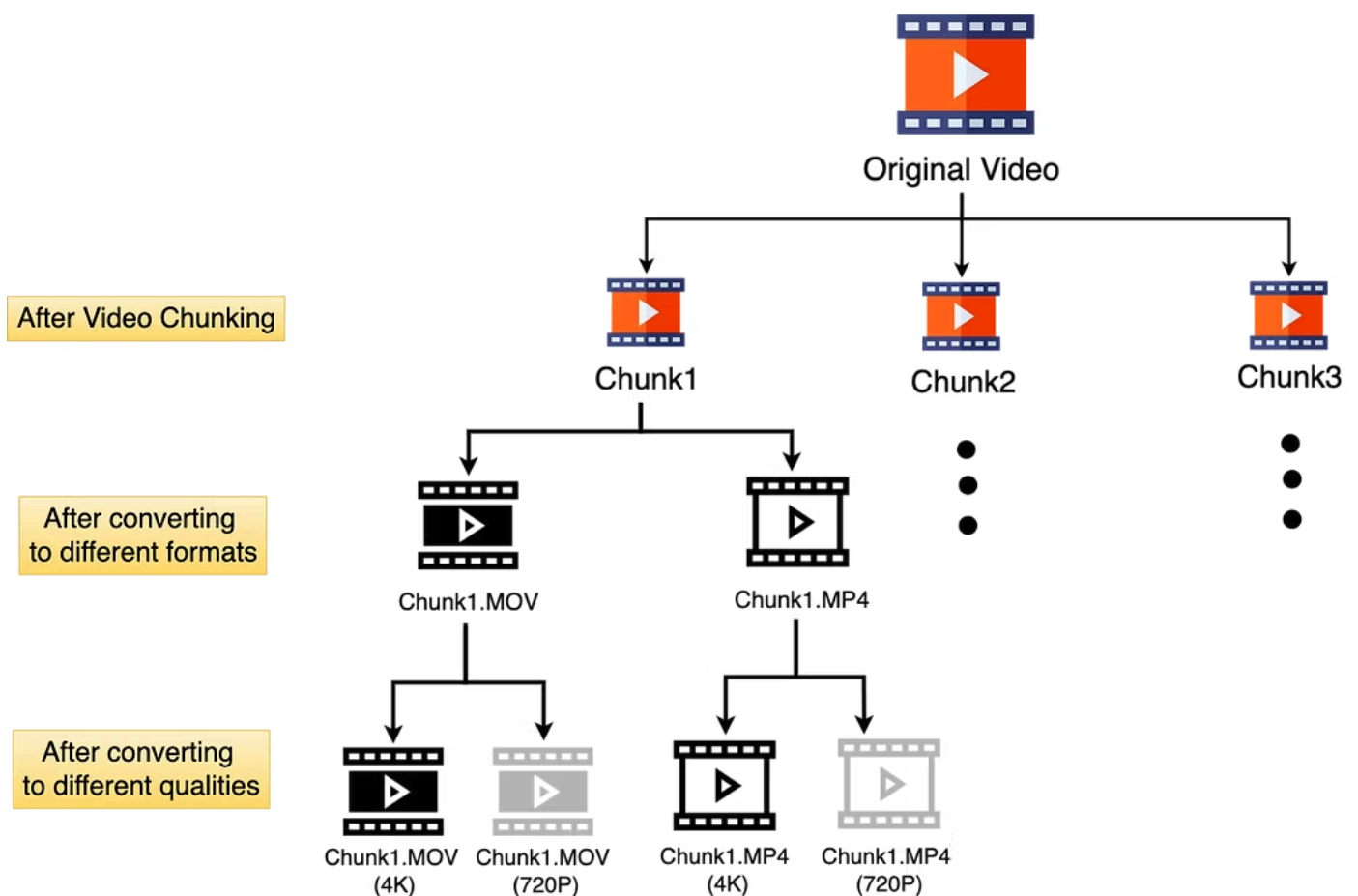
- If the connection is weak, they see 240P.
- If the connection is strong, they see 4K.



Steps Involved

1. **Retrieve the event from the Message Queue.**
2. **Use the chunk ID from the event to get the chunk from Object Storage.**
It then converts the chunk into different qualities/resolutions (e.g., 4K, 720P).
3. **Upload the converted chunks to Object Storage.**
4. **Create an event for each uploaded chunk and add it to the Message Queue.**
The event should include the chunk ID.

The steps above, when visualized, look like this:

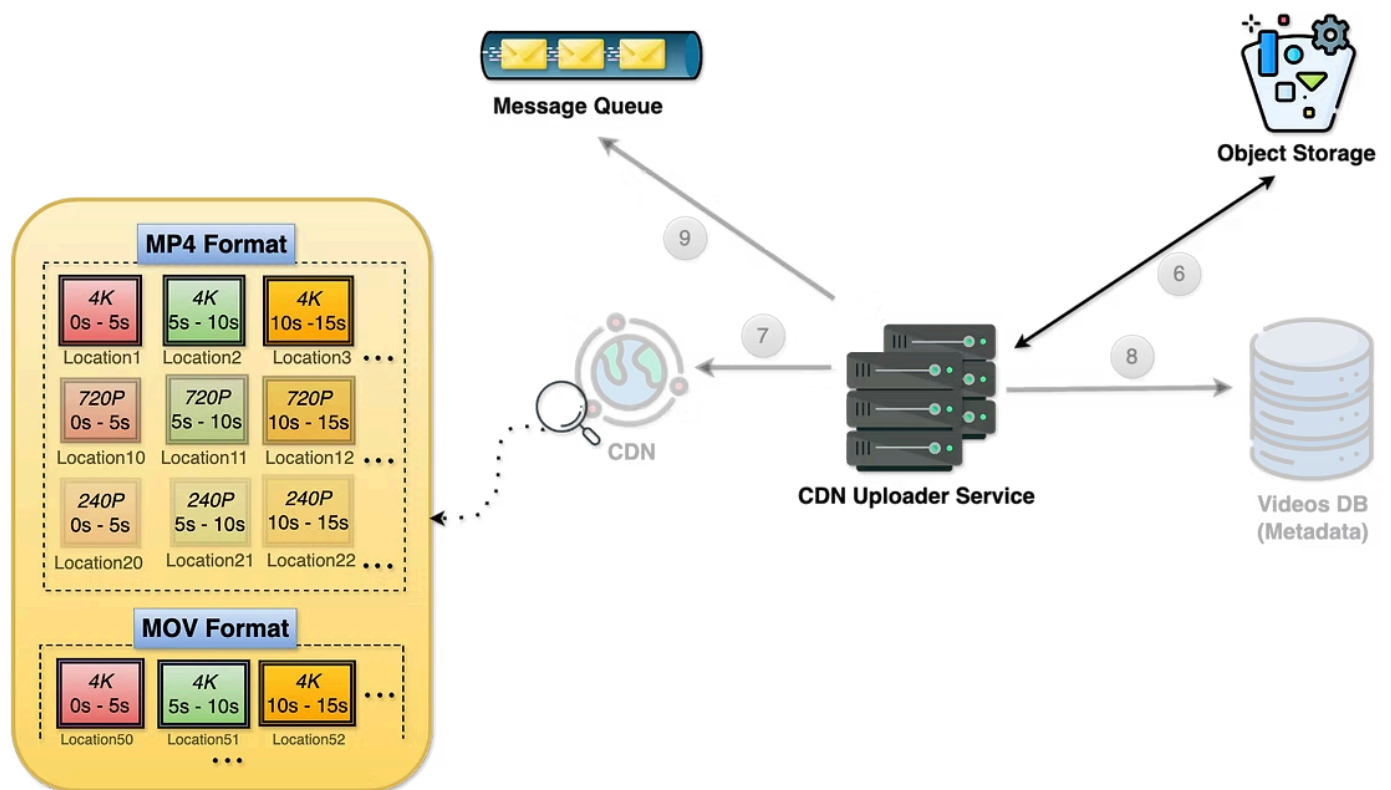


CDN Uploader Service

Finally, the CDN Uploader Service comes into play.

Note: The numbers 6, 7, 8, and 9 correspond to the high-level design diagram covered in the previous part.

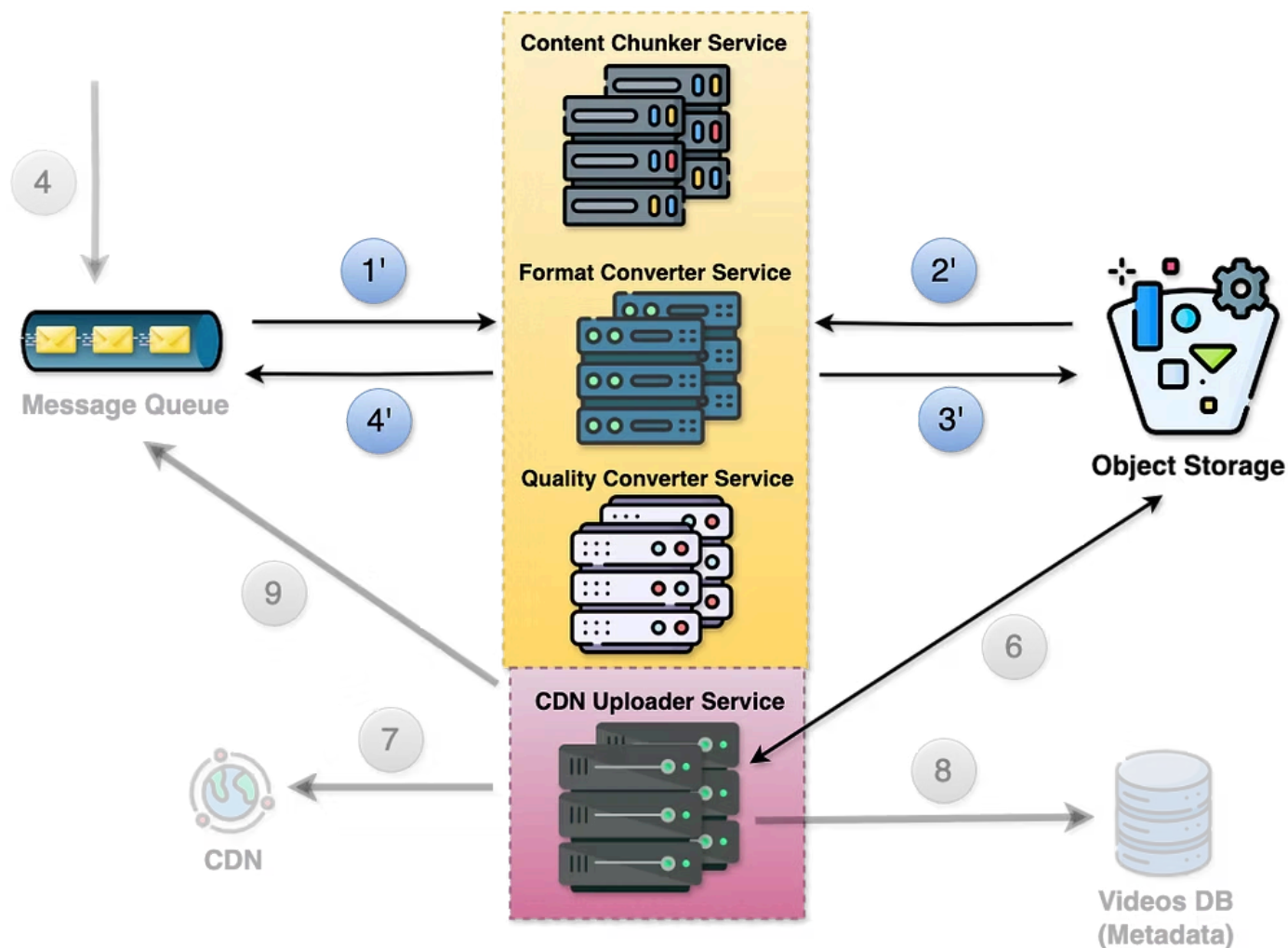
The CDN Uploader Service retrieves these resulting chunks (in different formats and qualities) from Object Storage and uploads them to the CDN. This is necessary because accessing videos from CDN is much faster.



Each chunk uploaded to the CDN has a location. The CDN Uploader Service also saves these locations in the Videos DB.

Ultimately, since the video processing is complete, the CDN Uploader Service adds an event to the Message Queue to signal that the video processing is done.

This is how the overall flow looks like:



DEEP DIVE INSIGHTS:

DEEP DIVE INSIGHTS: Database Selection

In order to decide the DB type, here are some general guidelines that you can follow. However, it's not always black and white — a lot depends on the project needs.

Guideline	Recommendation
When you need fast data access	Prefer NoSQL
When the scale is too large	NoSQL performs better
When the data fits into a fixed structure	Prefer SQL
When the data doesn't fit into a fixed structure	Choose NoSQL

Guideline	Recommendation
If you have complex queries to execute on your data	Use SQL
If you have simpler queries	NoSQL works well
If your data changes frequently or will evolve over time	NoSQL supports flexible structure

Database Decision Table

Database	Deciding Factors	Decision
VideosDB	- High Scale: Millions of YouTube videos are uploaded and watched every day. Handling such a scale of writes and reads makes NoSQL preferable. - Fast Access: Low latency is required due to non-functional requirements. NoSQL suits better for high-scale, low-latency systems. - Simple Query Pattern: When a user clicks on a video, the server returns video metadata along with the manifest file. Reading video metadata by video ID is well-handled by NoSQL.	NoSQL

DEEP DIVE INSIGHTS: Data Modeling

VideosDB Schema



Videos DB
(Metadata)

```
{
  "videoId": "unique_video_id",

  // Technical Metadata
  "format": "mp4",
  "duration": "length_of_video_in_s",
  ...

  // General Metadata
  "creatorId": "creator_id",
  "title": "video_title",
  "description": "video_description_text",
  ...

  // Content Delivery Metadata
  "cdnUrls" : {
    "MP4": {
      "4K": ["chunkurl1", "chunkurl2", ...],
      "720P": ["chunkurl1", "chunkurl2", ...],
      ...
    }
    "MOV": {
      "4K": ["chunkurl1", "chunkurl2", ...],
      "720P": ["chunkurl1", "chunkurl2", ...],
      ...
    }
    ...
  }
}
```

Attribute	Details
Database Type	NoSQL
Common Queries	Reading video metadata by videoId . This occurs when the user clicks on a video to watch the content, and the server returns back the video metadata along with the manifest file.
Indexing	videoId

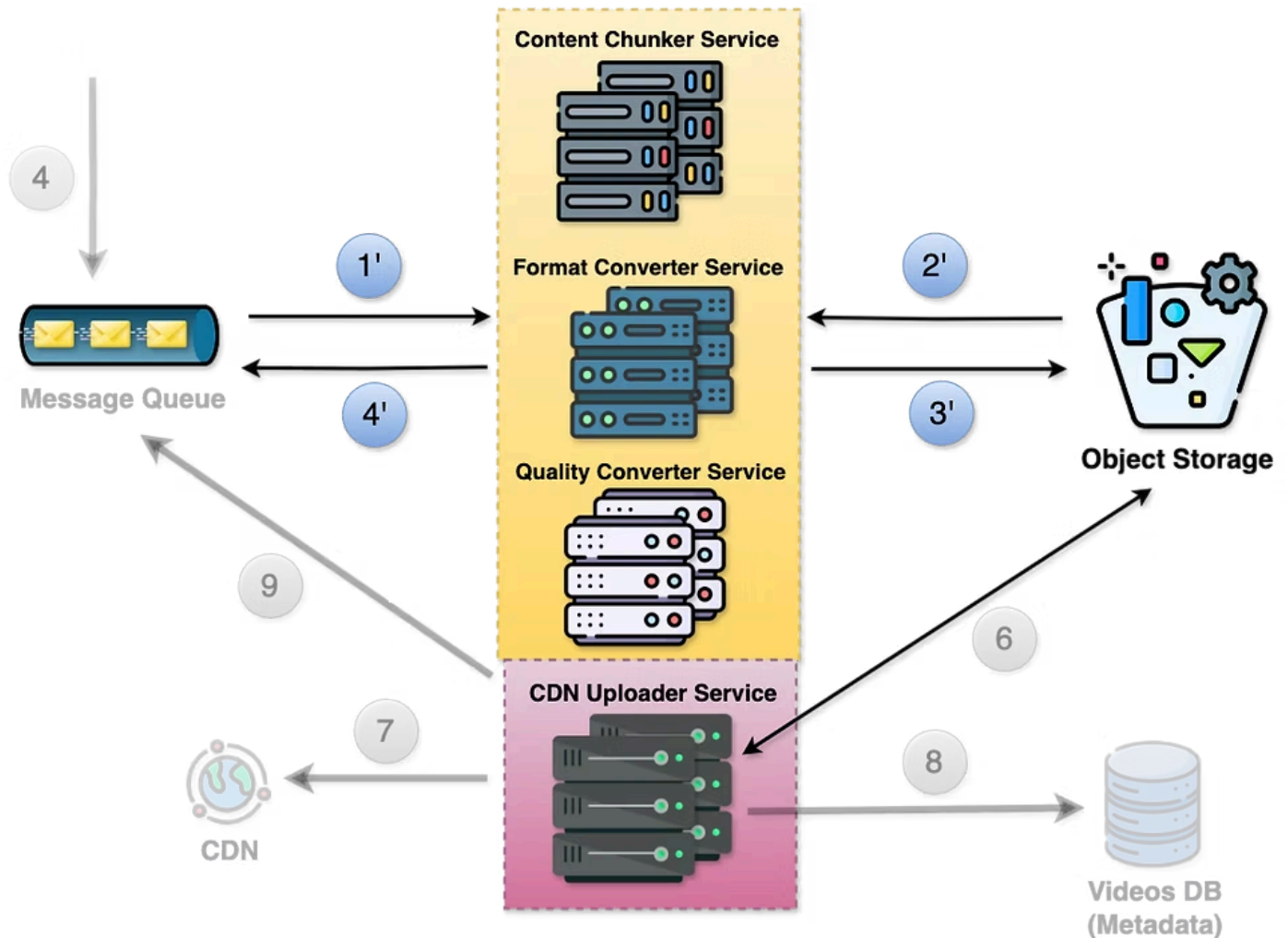
Note:

Because we have this common query to grab video metadata by **videoId**, we create an index on the **videoId** field. This sets a shortcut to quickly find the data by **videoId**. |

DEEP DIVE INSIGHTS: HLS Encoding

Workflow Engine Steps: Our Content Processor consists of four main steps:

1. **Content Chunking**
2. **Format Conversion**
3. **Quality Conversion**
4. **CDN Upload**



We also learned about HLS and how it enables adaptive streaming, which allows streaming different formats for different devices and different qualities based on internet speed.

Encoding for HLS

Another thing to know about HLS: whenever we are streaming videos with HLS, the video chunks need to be encoded in a certain way.

What is Encoding?

Encoding simply means turning the video into a stream of 0s and 1s. There are different ways to do this, and each way creates a different version (version = different representation of 0s and 1s) of the same video.

For HLS to work, the video needs to be encoded using specific standards like **H.264** or **H.265**.

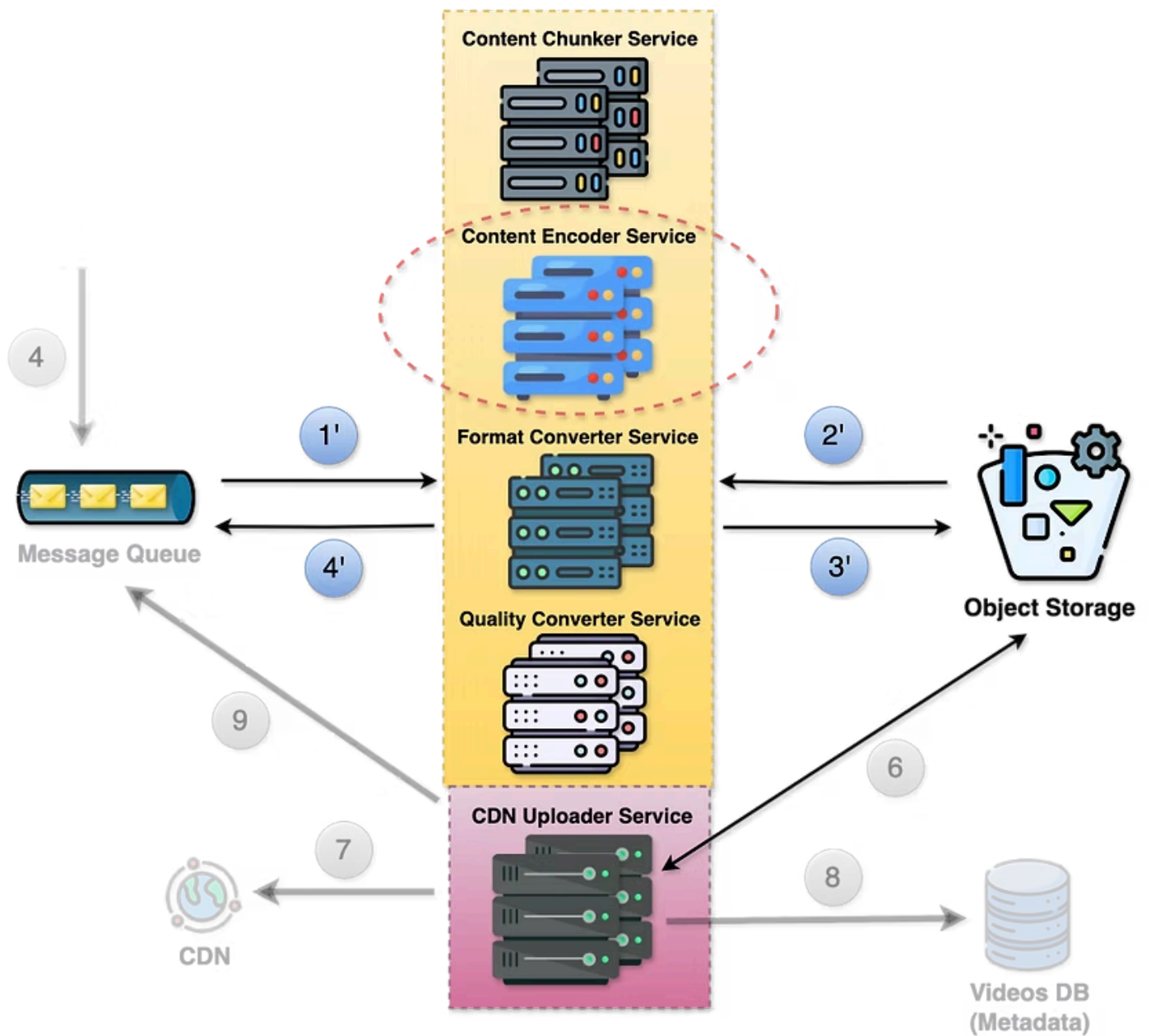
Current Workflow Problem

But have we ever included encoding in our process? Our current workflow doesn't have an encoding step. So, how should we tackle this problem?

Solution

By introducing the encoding step in the workflow engine. Here's how we can do it:

1. **Content Chunking**
2. **Encoding:** Encode each chunk in **H.264/H.265** standard.
3. **Format Conversion:** Convert each encoded chunk into different formats (e.g., **MP4**, **MOV**).
4. **Quality Conversion:** Convert each format into different qualities (e.g., **4K**, **720P**).
5. **CDN Upload:** Upload the final chunks to the CDN.



The introduction of the encoding step ensures that our workflow aligns with what HLS needs, enabling adaptive streaming.